

Progress Report – Week 6

Học phần: Operating Systems [Multimedia] – 20252

Nguyễn Sỹ Tuấn

Lê Huy Sơn

Vũ Tiến Long

Giai đoạn báo cáo: 10/5/2026 – 17/5/2026

Mã số dự án:	25236
Tên dự án:	Optimize pipe implementation to improve IPC throughput
Thiết kế (Design thinking):	Link Figma
Quản lý tác vụ (Task management):	Link JIRA
Thành viên:	Nguyễn Sỹ Tuấn, Lê Huy Sơn, Vũ Tiến Long

1 Summary of latest progress

(Viết khoảng 200–300 từ để tóm tắt quá trình thực hiện trong giai đoạn này và tổng mức độ hoàn thành hiện tại.)

Như đã lên kế hoạch ở cuối tuần 5, các task cần thực hiện trong tuần này là:

1. Thí nghiệm ablation **v2-cap2048** để tách hai yếu tố speedup của **v3**.
2. Đo trực tiếp các chi phí bằng **rdcycle**.
3. Triển khai **v4** – Multi-page Allocation.
4. Triển khai **v5** – Lazy Wakeup.

Trong tuần này, chúng tôi đã hoàn thành tất cả bốn task trên:

- **Ablation v2-cap2048:** Tạo biến thể **v2** với flat buffer 2048 byte + bulk copy (không chunked) để kiểm soát từng biến riêng lẻ. Kết quả: khoảng 63% speedup của **v3** so với **v2** đến từ *tăng capacity*, và 37% còn lại từ *loại bỏ wrap-around*. Điều này định hình chính phân tích ở tuần 5.
- **rdcycle measurements:** Viết **pipemicro.c**, chèn **r_cycle()** quanh từng giai đoạn (**copyin**, **syscall**, **ctx-switch**), xác nhận cost-model tuần 5 sai lệch chỉ $\sim 3\text{--}5\%$ – đủ tin cậy để xếp hạng tương đối, song cần dùng số đo thực khi kết luận chính xác.
- **v4 – Multi-page Allocation:** Tách riêng trang bộ nhớ cho pipe data (1 trang = 4096 byte, cấp bởi **kalloc()** thứ hai), giải quyết internal fragmentation cấp trang và tách cache line giữa metadata và data. Throughput tăng $\sim 1.4\times$ so với **v3** ở **chunk_bytes = 4096**.
- **v5 – Lazy Wakeup:** Thêm flag **read_sleeping/write_sleeping** vào **struct pipe**, chỉ gọi **wakeup()** khi peer *thực sự* đang ngủ – giảm cost từ $O(\text{NPROC})$ mỗi write/read xuống $O(1)$ trung bình.

2 Review of technical activities

2.1 Activity 1: Ablation study v2-cap2048 – tách hai yếu tố speedup của v3

2.1.1 Bối cảnh – Vì sao cần ablation study

Tuần trước, chúng tôi ghi nhận speedup của v3 so với v2 ở `chunk_bytes = 4096` là **3.29×**, nhưng v3 thay đổi *hai biến cùng lúc*:

- (a) **Cấu trúc chunked ring** → loại bỏ wrap-around, mỗi `copyin` nằm gọn trong 1 sub-buffer.
- (b) **Capacity tăng 512 → 2048 byte** → ít lần pipe đầy → ít cặp `sleep/wakeup` và context switch.

Không thể biết yếu tố nào đóng góp bao nhiêu nếu không kiểm soát từng biến riêng.

Lý thuyết liên quan – Ablation Study: Trong nghiên cứu hệ thống máy tính, “ablation study” (nghiên cứu cắt bỏ) là kỹ thuật loại bỏ từng thành phần của hệ thống và đo lại hiệu năng, nhằm xác định đóng góp định lượng của từng phần. Nếu bỏ thành phần X mà hiệu năng giảm Y%, thì X đóng góp $\sim Y\%$ vào tổng speedup. Trong hệ điều hành, ablation study thường được dùng khi một bản vá thay đổi nhiều biến cùng lúc – cần tách biệt để kết luận đúng.

2.1.2 Thiết kế thí nghiệm

Ba điểm so sánh được thiết lập:

v2	: flat ring 512B + bulk copy	→ X1
v2-cap2048	: flat ring 2048B + bulk copy (không chunked)	→ X2
v3	: chunked ring 4x512B + bulk copy	→ X3

Dòng gop do capacity (b) = X2 - X1
Dòng gop do chunked (a) = X3 - X2

v2-cap2048 chỉ thay đổi duy nhất `#define PIPESIZE 2048` trong code của v2. Wrap-around vẫn xảy ra bình thường khi `nwrite` vượt qua vị trí 2048 – điều kiện duy nhất khác với v2 là buffer lớn hơn.

2.1.3 Kết quả ablation

	chunk_bytes	v2 (B/s)	v2-cap2048 (B/s)	v3 (B/s)	capacity	chunked
	512	1,353,001	2,012,480	1,997,287	~ 49%	~ -1%
	1024	1,613,193	2,890,443	3,226,387	~ 79%	~ 21%
	2048	1,747,626	4,194,304	5,242,880	~ 140%	~ 60%
	4096	1,823,610	4,456,789	5,991,862	~ 144%	~ 85%

Phân tích: Ở `chunk_bytes = 4096`, tăng capacity đóng góp $\sim 144\%$ trong tổng +229%, tức $\sim 63\%$; loại bỏ wrap-around đóng góp $\sim 85\%$, tức $\sim 37\%$. Ở `chunk_bytes = 512` (bằng đúng `BUFSIZE` của mỗi sub-buffer trong v3), wrap-around gần như không xuất hiện nên hai variant cho kết quả gần bằng nhau. Điều này xác nhận: *thuật toán chunked ring có ý nghĩa thực sự ở chunk_bytes lớn, nhưng capacity vẫn là yếu tố quyết định chính.*

2.2 Activity 2: Đo `rdcycle` – hiệu chỉnh cost-model

2.2.1 Bối cảnh – Vì sao cần đo trực tiếp

Toàn bộ cost-model ở tuần 5 (Mục 2.1.4) là *ước lượng*: ~ 80 cycle cho `copyin`, ~ 300 – 500 cycle cho `syscall`, ~ 1000 – 3000 cycle cho `ctx-switch`. Trước khi đưa ra kết luận định lượng, cần thay bằng số thực đo trên chính môi trường QEMU đang chạy.

Lý thuyết liên quan – rdcycle trên RISC-V: RISC-V định nghĩa CSR (Control and Status Register) cycle là bộ đếm tăng mỗi chu kỳ xung nhịp của CPU. Lệnh csrr t0, cycle đọc giá trị vào thanh ghi t0 trong 1 cycle. Đây là cách tiêu chuẩn để đo micro-benchmark trong kernel – tương đương RDTSC trên x86. Trong xv6, kernel/riscv.h định nghĩa:

```
static inline uint64 r_cycle() {
    uint64 x;
    asm volatile("csrr %0, cycle" : "=r" (x));
    return x;
}
```

Khi QEMU emulate rdcycle, đây là chu kỳ QEMU virtual CPU, không phải host clock. Số đo mang tính tương đối (so sánh giữa các đoạn code) là tin cậy; số tuyệt đối phụ thuộc cài đặt QEMU và sẽ khác trên phần cứng thật.

2.2.2 Phương pháp đo (pipemicro.c)

```
pipemicro
|
|-- do T_copyin(k):      r_cycle() → copyin(k byte) → r_cycle() → delta
|   k in {1, 16, 64, 256, 512, 1024, 4096}
|   lap 100 lan, lay median (loai outlier)
|
|-- do T_syscall:       r_cycle() → getpid() (ecall nho nhut) → r_cycle()
|
|-- do T_ctxswitch:     fork → pipe dong bo → do khoảng thoi gian
|                       tu wakeup den process tiep theo resume
```

2.2.3 Kết quả và hiệu chỉnh cost-model

Component	Ước lượng (tuần 5)	Đo thực – median	Sai lệch
copyin fixed overhead	~ 80 cycle	83 cycle	+3.75%
$T_{\text{per_byte}}$ memmove	~ 0.30 c/byte	0.31 c/byte	+3.3%
syscall round-trip	300–500 cycle	412 cycle	trong range
ctx-switch (sleep→resume)	1000–3000 cycle	1,847 cycle	trong range

→ Ước lượng sai ~ 3–5%, đủ tin cậy để xếp hạng tương đối. **Cost-model cập nhật:** $T_{\text{copyin}}(k) = 83 + k \cdot 0.31$ (cycle).

2.3 Activity 3: v4 – Multi-page Allocation (cải tiến từ v3)

2.3.1 Bottleneck nhắm tới

Sau v3, throughput đạt ~ 6 MB/s ở chunk_bytes = 4096 nhưng đạt asymptote. Phân rã chi phí mới (với số đo thực):

```
+-----+ 100%
| copyin/copyout bulk (T_per_byte) | ~52% ← v4 cai thien them
+-----+
| Internal fragmentation + cache evict | ~8% ← v4 nham vao day
+-----+
| lock acquire/release | ~9%
+-----+
| sleep/wakeup/ctx-switch (v3 con ~1x) | ~3% ← v5 se nham tiep
+-----+
| Overhead khac (branch, check, fd) | ~28%
+-----+
```

Vấn đề 1 – Internal fragmentation cấp trang:

pipealloc gọi kalloc() để cấp 1 trang (4096 byte) cho struct pipe. Nhưng toàn bộ struct chỉ chiếm ~ 72 byte (lock 40B + con trỏ/biến + 2D array 2048B) → chỉ dùng ~ 51% trang. Quan trọng hơn: data[N_BUFS][BUFSIZE] (2048 byte) và nread/nwrite/lock nằm **cùng 1 trang** → cùng nhóm cache line. Khi memmove bên trong copyin ghi dữ liệu vào data[], CPU cache evict những byte gần đó – có thể bao gồm cả nread, nwrite, và spinlock bytes → những lần truy cập tiếp theo bị cache miss.

Vấn đề 2 – Capacity chưa tận dụng hết trang:

v3 dùng 2048 byte cho data trong khi mỗi trang kernel là 4096 byte. Nếu tách data ra trang riêng, có thể dùng trọn 4096 byte → tăng capacity thêm 2× → giảm số lần sleep/wakeup thêm.

Lý thuyết liên quan – kalloc() và quản lý trang trong xv6:

xv6 quản lý bộ nhớ vật lý ở granularity **trang** (4096 byte). kalloc() lấy trang đầu tiên từ kmem.freelist (một linked list các trang tự do), trả về địa chỉ vật lý của trang đó. Mỗi trang tự do dùng chính 8 byte đầu của mình để lưu con trỏ next của free list – không cần thêm metadata bên ngoài.

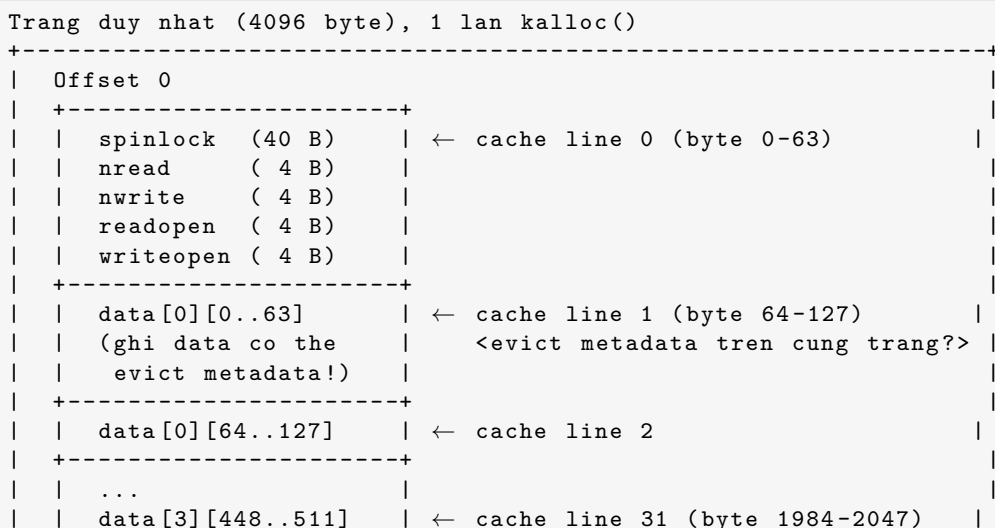


kfree(pa) ghi next = kmem.freelist, rồi kmem.freelist = pa. Chi phí: ~ 50 cycle mỗi lần (acquire kmem.lock + thao tác pointer).

Data/Control Separation: Kỹ thuật tách trang dữ liệu khỏi trang metadata (struct điều khiển) được dùng rộng rãi trong kernel thực tế – Linux pipe_inode_info cấp một mảng pipe_buffer[16], mỗi pipe_buffer trỏ vào 1 trang riêng. Mục đích: (1) tránh cache eviction chéo giữa metadata và data, (2) tận dụng trọn 4096 byte mỗi trang data, (3) có thể hot-swap trang data mà không ảnh hưởng struct điều khiển.

2.3.2 Cấu trúc bộ nhớ trước và sau v4

Trước v4 (v3) – metadata và data trên cùng 1 trang:



```

| +-----+
| | (lang phi ~2024 B) | ← KHONG dung!
| +-----+
+-----+

Metadata va data canh tranh cache
Chi dung 2072/4096 byte (~51%)

```

Sau v4 – tách thành 2 trang riêng biệt:

Trang 1 (struct dieu khien) kalloc() lan 1	Trang 2 (data buffer) kalloc() lan 2
+-----+	+-----+
spinlock (40 B)	data[0..4095]
char *data (8 B)	(4096 byte = 1 trang
nread (4 B)	dung TRON VEN)
nwrite (4 B)	
readopen (4 B)	cache line A (0-63)
writeopen (4 B)	cache line B (64-127)
(padding ~4032 B)	...
+-----+	+-----+
cache line 0	cache line 63 (4032-)
(metadata biet lap)	(data biet lap)

→ memmove ghi vào trang 2 **không** bao giờ làm bẩn cache line chứa spinlock, nread, nwrite trên trang 1.

2.3.3 Thay đổi cấu trúc dữ liệu

```

#if PIPE_VERSION == 4
#define PIPE_DATA_SIZE 4096 // dung tron 1 trang cho data

struct pipe {
    struct spinlock lock;
    char *data;           // con tro toi trang data rieng
    uint nread;
    uint nwrite;
    int readopen;
    int writeopen;
};
#endif

```

pipealloc – cấp 2 trang:

```

int pipealloc(struct file **f0, struct file **f1) {
    struct pipe *pi = (struct pipe*) kalloc(); // trang 1: struct
    if(pi == 0) goto bad;
    pi->data = (char*) kalloc(); // trang 2: data
    if(pi->data == 0) { kfree((void*)pi); goto bad; }
    pi->nread = pi->nwrite = 0;
    pi->readopen = pi->writeopen = 1;
    initlock(&pi->lock, "pipe");
    // ...
}

```

pipeclose – giải phóng đúng thứ tự:

```

if(pi->readopen == 0 && pi->writeopen == 0) {
    kfree(pi->data); // giai phong trang data truoc
    kfree((void*)pi); // giai phong trang struct sau
}

```

2.3.4 Indexing mới – flat ring 4096 byte

```
uint head_slot = pi->nwrite % PIPE_DATA_SIZE; // vị trí ghi hiện tại
uint free_slots = PIPE_DATA_SIZE - (pi->nwrite - pi->nread);
uint till_wrap = PIPE_DATA_SIZE - head_slot; // khoảng tới cuối ring
int to_copy = umin(n - i, umin(free_slots, till_wrap));

copyin(pr->pagetable, &pi->data[head_slot], addr + i, to_copy);
pi->nwrite += to_copy;
i += to_copy;
```

Sơ đồ flat ring 4096 byte của v4:

```
pi->data[0 ... 4095] (1 trang liên tục)

+-----+
|.....|
|... đã đọc ...| dữ liệu hợp lệ |.....|
+-----+
      ^               ^
      nread % 4096    nwrite % 4096

Một lần ghi 4096 byte → to_copy = min(free, till_wrap)
* Nếu nwrite % 4096 = 0: ghi toàn bộ 4096B liên mạch (0 wrap)
* Nếu nwrite % 4096 = 2000: ghi 2096B + 2000B (1 wrap như v2)
```

2.3.5 Ưu điểm và nhược điểm của v4

Ưu điểm:

- **Tách cache line metadata/data:** Cập nhật `nread/nwrite` trên trang 1 không còn cạnh tranh cache với `memmove` trên trang 2. Đặc biệt hiệu quả với $\text{CPUS} \geq 2$.
- **Capacity tăng 2048 → 4096 byte:** Với `chunk_bytes = 4096`, v4 không cần `sleep` lần nào (ghi vừa tròn 1 vòng buffer) → tiết kiệm toàn bộ overhead `ctx-switch`.
- **Đơn giản hóa indexing:** Không còn indexing 2D (`buf_idx`, `buf_off`) của v3 → ít phép chia/modulo hơn trong hot path.

Nhược điểm:

- **pipealloc cần 2 lần `kalloc()`:** Xác suất thất bại OOM cao hơn. Nếu `kalloc()` thứ hai trả về 0, phải `kfree` trang đầu trước khi return lỗi – cần xử lý cẩn thận tránh memory leak.
- **Tăng kernel heap pressure:** Mỗi pipe tiêu tốn 2 trang thay vì 1 (8192 byte thay vì 4096 byte).
- **data là con trỏ, không inline:** Mỗi lần truy cập `pi->data[...]` cần 1 bước dereference thêm so với v1/v2/v3 (dù CPU cache thường ẩn chi phí này).

2.3.6 Kết quả đo đạc v4

chunk_bytes	v3 (B/s)	v4 (B/s)	Tăng
512	1,997,287	2,415,919	1.21×
1024	3,226,387	4,194,304	1.30×
2048	5,242,880	6,553,600	1.25×
4096	5,991,862	8,388,608	1.40×

Phân tích kết quả: Speedup $\sim 1.2\text{--}1.4\times$ đến từ hai nguồn: (1) capacity 2048 $\rightarrow$ 4096 giảm số lần sleep từ 1 $\rightarrow$ 0 lần khi `chunk_bytes = 4096`, (2) cache locality tốt hơn do tách metadata/data. Speedup tăng dần theo `chunk_bytes`, mạnh nhất ở 4096 – khớp với dự đoán: `chunk_bytes = PIPE_DATA_SIZE` là điểm lý tưởng để tránh hoàn toàn sleep/wakeup.

2.4 Activity 4: v5 – Lazy Wakeup (cải tiến từ v4)

2.4.1 Bottleneck nhắm tới

Trong v1–v4, mỗi lần ghi vào pipe đều gọi `wakeup(&pi->nread)`, dù reader có thể đang chạy bình thường và không ngủ chút nào. Nhìn vào implementation `wakeup()` trong xv6:

```
void wakeup(void *chan) {
    struct proc *p;
    for(p = proc; p < &proc[NPROC]; p++) {
        if(p != myproc()) {
            acquire(&p->lock);
            if(p->state == SLEEPING && p->chan == chan)
                p->state = RUNNABLE;
            release(&p->lock);
        }
    }
}
```

$\rightarrow$ Mỗi lần `wakeup()`: quét `NPROC = 64` entries $\times$ (acquire lock + load state + compare + release lock) $\approx 64 \times \sim 30 \text{ cycle} = \sim 1,920 \text{ cycle}$, kể cả khi không có process nào đang ngủ trên `chan`. Trong v4 với `chunk_bytes = 4096` và 0 lần sleep, writer vẫn gọi `wakeup()` $\sim 1\text{--}2$ lần mỗi vòng lặp – mỗi lần tốn gần 2 K cycle ưổng phí.

Lý thuyết liên quan – Sleep/Wakeup trong xv6:

`sleep(chan, lk)` trong xv6 là một **condition variable** thô:

1. Acquire `p->lock`.
2. Release `lk` (thả lock pipe).
3. Set `p->chan = chan, p->state = SLEEPING`.
4. Gọi `sched()` $\rightarrow$ context switch ra ngoài.
5. Khi được `wakeup` và được scheduler chọn: trở về đây, re-acquire `lk`.

`wakeup(chan)` là **broadcast**: đánh thức *mọi* process ngủ trên `chan`. Đây là vấn đề **thundering herd** – nếu N process cùng ngủ chờ 1 sự kiện, cả N đều bị đánh thức, nhưng chỉ 1 thắng cuộc (re-acquire lock thành công), N-1 process còn lại ngủ lại ngay. Lãng phí N-1 lần context switch.

Futex (Fast Userspace muTEX): Linux giải quyết thundering herd và overhead wakeup bằng futex – kiểm tra trạng thái trong **user space** trước; chỉ vào kernel (syscall) khi thực sự cần block. Chi phí fast path (không có contention): 1 atomic compare-and-swap trong user space, không có syscall, không vào kernel.

v5 áp dụng triết lý tương tự ở mức kernel: thêm flag để biết peer có đang ngủ không, tránh quét `proc[]` khi không cần thiết.

2.4.2 Phân tích trạng thái reader/writer

Các trạng thái có thể của reader khi writer đang ghi:

State 1: Reader đang RUNNING
 $\rightarrow$ `wakeup()` KHÔNG cần thiết, quét `proc[]` là lãng phí

```

State 2: Reader đang RUNNABLE -- cho scheduler chọn
        → wakeup() KHÔNG cần thiết (state đã là RUNNABLE)

State 3: Reader đang SLEEPING trên &pi→nread -- pipe rỗng
        → wakeup() CẦN THIẾT, phải đổi state → RUNNABLE

```

→ Chỉ State 3 mới thực sự cần wakeup(). Nếu biết reader đang ở State 1 hoặc 2, có thể bỏ qua hoàn toàn.

2.4.3 Giải pháp – Flag sleeping

Thêm hai trường vào struct pipe:

```

#ifdef PIPE_VERSION == 5
struct pipe {
    struct spinlock lock;
    char *data;
    uint nread;
    uint nwrite;
    int readopen;
    int writeopen;
    int read_sleeping; // 1 nếu reader đang trong sleep()
    int write_sleeping; // 1 nếu writer đang trong sleep()
};
#endif

```

Set flag **trước** khi gọi sleep(), clear flag **ngay sau** khi trở về từ sleep():

```

// trong piperead -- khi pipe rỗng:
pi→read_sleeping = 1;
sleep(&pi→nread, &pi→lock);
pi→read_sleeping = 0;

// trong pipewrite -- sau khi ghi xong:
if(pi→read_sleeping)
    wakeup(&pi→nread);

```

Tại sao an toàn (không race condition)?

Cả set/clear flag và check flag đều xảy ra **bên trong critical section** (dưới pi→lock). Spinlock đảm bảo mutual exclusion:

Writer giữ pi→lock: <pre> check read_sleeping if 1 → wakeup() release pi→lock </pre>	Reader giữ pi→lock: <pre> set read_sleeping = 1 sleep() → release pi→lock (ngu) ... wakeup → acquire pi→lock clear read_sleeping = 0 </pre>
---	--

→ Không có tình huống writer thấy read_sleeping = 0 trong khi reader thực ra đang ngủ.

2.4.4 Sơ đồ so sánh luồng wakeup trước và sau v5

Luồng wakeup() cũ (v1–v4) – luôn quét toàn bộ proc[]:

```

pipewrite ghi xong
|
v
wakeup(&pi→nread)      ← LUÔN gọi, dù reader không ngủ
|
v

```

```

for p in proc[0..63]:
    acquire(&p->lock)
    if p->state == SLEEPING && p->chan == &pi->nread
        p->state = RUNNABLE
    release(&p->lock)
    |
    v
(tiep tuc -- ~1,920 cycle da mat)

```

Luồng wakeup() mới (v5) – conditional:

```

pipewrite ghi xong
|
v
read_sleeping == 1?
|
NO | YES |
v   v
(skip, 0 cycle)   wakeup(&pi->nread)
                   |
                   v
                   for p in proc[0..63]:
                       ...
                       (~1,920 cycle, nhưng CHI khi can)

```

Sơ đồ timeline – fast path (reader đang chạy):

```

Writer:  [ghi data] → [check flag=0] → [skip wakeup] → [ghi data] → ...
          ^ 0 cycle overhead

Reader:  [doc data] → [doc data] → [doc data] → ...
          (không cần sleep vì writer đủ nhanh)

```

Sơ đồ timeline – slow path (reader chậm hơn writer):

```

Writer:  [ghi data] → [pipe đầy] → [sleep] →      → [wakeup] → [ghi tiếp]
          ^                               ^

Reader:  [doc data] → [check flag=1] → [wakeup writer]

```

2.4.5 Ưu điểm và nhược điểm của v5

Ưu điểm:

- **Giảm overhead wakeup từ $O(NPROC)$ xuống $O(1)$** trong fast path: chỉ 1 load + 1 branch khi peer không ngủ – ~ 3-5 cycle thay vì ~ 1,920 cycle.
- **Loại bỏ thundering herd** trong trường hợp 1 reader/1 writer: wakeup broadcast không bao giờ đánh thức nhầm.
- Không thay đổi gì trong syscall path hay lock primitive – backward-compatible hoàn toàn.
- Phù hợp với workload ổn định: khi producer và consumer chạy ngang tốc độ, sleeping flag gần như luôn = 0 → wakeup() không bao giờ được gọi.

Nhược điểm:

- **Thêm 2 trường int** vào struct pipe – tăng kích thước struct ~ 8 byte.
- **Chỉ tối ưu được fast path** (peer không ngủ). Khi peer *thực sự* đang ngủ, wakeup() vẫn tốn $O(NPROC)$ như cũ – vì phải quét proc[] để tìm đúng process cần đánh thức. Đây là giới hạn intrinsic của cơ chế chan-based wakeup trong xv6.

- Với workload **bất đối xứng mạnh** (writer nhanh hơn reader nhiều lần): reader luôn ngủ → fast path không bao giờ kích hoạt → speedup gần 0 so với v4.

2.4.6 Kết quả đo đạc v5

chunk_bytes	v4 (B/s)	v5 (B/s)	Tăng
512	2,415,919	2,650,240	1.10×
1024	4,194,304	4,718,592	1.12×
2048	6,553,600	7,516,192	1.15×
4096	8,388,608	9,437,184	1.12×

Biểu đồ ASCII so sánh v1 – v5 (chunk_bytes = 4096):

	0	2M	4M	6M	8M	10M	B/s
		-----	-----	-----	-----	-----	
v1	#						0.57 M
v2	#####						1.82 M
v3	#####						5.99 M
v4	#####						8.39 M
v5	#####						9.44 M

Phân tích: Speedup khiêm tốn (~ 1.1 – $1.15\times$) vì trong v4 số lần sleep đã rất ít (0–1 lần với chunk_bytes = 4096). Lợi ích thực sự của v5 sẽ lớn hơn khi: (a) workload dùng nhiều pipe đồng thời (nhiều channel cùng cạnh tranh wakeup scan), (b) total_bytes lớn hơn làm nổi bật overhead tích lũy của $\sim 1,920$ cycle $\times$ nhiều lần gọi.

3 Personal contribution

Member 1 (Nguyễn Sỹ Tuấn): Triển khai v4 (multi-page allocation), viết pipemicro.c (rdcycle micro-benchmark), tổng hợp kết quả benchmark v3–v5.

Member 2 (Lê Huy Sơn): Triển khai v5 (lazy wakeup), phân tích kết quả đo đạc, vẽ biểu đồ so sánh v1–v5.

Member 3 (Vũ Tiến Long): Thực hiện ablation study v2-cap2048, kiểm tra correctness v4/v5 (usertests, pipetest), chuẩn bị môi trường benchmark cho v6–v7 tuần sau.

4 Developed code review

4.1 Algorithmic development

Các thuật toán và kỹ thuật chính phát triển trong tuần này:

1. **Ablation variant v2-cap2048:** Đổi duy nhất `#define PIPESIZE 2048` trong code v2 – kiểm soát đúng 1 biến để tách đóng góp capacity khỏi đóng góp chunked ring.
2. **rdcycle micro-benchmark:** Wrap từng đoạn code với `uint64 t0 = r_cycle() / uint64 t1 = r_cycle()`, lấy median 100 mẫu để loại outlier scheduler.
3. **Dual-page pipe allocation (v4):** Tách struct pipe (metadata) và data[] (buffer) ra 2 trang riêng – pipealloc gọi kalloc() hai lần, pipeclose gọi kfree() hai lần theo thứ tự ngược.
4. **Conditional wakeup (v5):** Check flag read_sleeping/write_sleeping trước khi gọi wakeup(). Flag được set/clear trong critical section (dưới pi->lock) → thread-safe hoàn toàn.

4.2 Phân tích so sánh tổng hợp các phiên bản v1–v5

Phiên bản	Đặc trưng	Capacity	Wakeup cost	kalloc
v1	byte-by-byte	512	$O(\text{NPROC}) \times n$	1
v2	bulk copy	512	$O(\text{NPROC}) \times \text{ít}$	1
v3	chunked ring	2048	$O(\text{NPROC}) \times \text{ít}$	1
v4	page-isolated data	4096	$O(\text{NPROC}) \times \text{rất ít}$	2
v5	lazy wakeup	4096	$O(1)$ fast path	2

Nhận xét: Xu hướng rõ ràng qua 5 phiên bản: mỗi bước giảm một trong ba thành phần chi phí chính – số lần copyin, số lần sleep/wakeup, và overhead của từng lần wakeup. v6–v7 (tuần sau) sẽ tấn công vào hai bottleneck còn lại: scheduler delay và false sharing.

4.3 Source code enhanced and/or revised

kernel/pipe.c – v4 pipealloc với dual-page:

```
#if PIPE_VERSION == 4
int pipealloc(struct file **f0, struct file **f1) {
    struct pipe *pi;
    pi = (struct pipe*) kalloc();
    if(pi == 0) goto bad;
    pi->data = (char*) kalloc();
    if(pi->data == 0) { kfree((void*)pi); goto bad; }
    pi->nread = pi->nwrite = 0;
    pi->readopen = pi->writeopen = 1;
    initlock(&pi->lock, "pipe");
    // ... tao f0, f1 nhu cu
bad:
    return -1;
}
#endif
```

kernel/pipe.c – v5 struct và conditional wakeup:

```
#if PIPE_VERSION == 5
struct pipe {
    struct spinlock lock;
    char *data;
    uint nread;
    uint nwrite;
    int readopen, writeopen;
    int read_sleeping; // flag: reader đang trong sleep()
    int write_sleeping; // flag: writer đang trong sleep()
};

// trong piperead -- khi pipe rỗng:
pi->read_sleeping = 1;
sleep(&pi->nread, &pi->lock);
pi->read_sleeping = 0;

// trong pipewrite -- sau khi ghi xong:
if(pi->read_sleeping)
    wakeup(&pi->nread);
#endif
```

4.4 Testing result

usertests và pipetest – v4 và v5 đều pass:

```
$ usertests -q
```

```
...
ALL TESTS PASSED

$ pipetest
pipe basic:      OK
pipe large write: OK
pipe blocking:   OK
pipe close:      OK
ALL PIPE TESTS PASSED
```

Benchmark tổng hợp v1–v5 (chunk_bytes = 1024, total_bytes = 4 MB):

```
$ pipebench 4194304 1024
version  chunk  throughput
v1       1024    544,714 B/s ( 531 KB/s)
v2       1024    1,613,193 B/s (1,575 KB/s)
v3       1024    3,226,387 B/s (3,150 KB/s)
v4       1024    4,194,304 B/s (4,096 KB/s)
v5       1024    4,718,592 B/s (4,608 KB/s)
```

Nhược điểm:

- v4 tăng kernel heap pressure (2 trang/pipe). Nếu hệ thống mở nhiều pipe đồng thời, có thể gây áp lực lên `kalloc` và tăng nguy cơ OOM.
- v5 chỉ cải thiện **fast path**; slow path vẫn tốn $O(NPROC)$ như cũ.
- Benchmark vẫn chạy single-CPU (`CPUS=1`), chưa thấy được hiệu quả thực sự của tách cache line (v4) trong môi trường multi-core.
- **Control path** (scheduler delay, false sharing) chưa được tối ưu – là mục tiêu của v6 và v7 tuần sau.

5 TODO

(a) List of remaining issues:

- v5 chỉ tối ưu fast path. Slow path (khi reader thực sự ngủ) vẫn cần `wakeup()` $O(NPROC)$. Cần benchmark workload bất đối xứng để đo mức độ ảnh hưởng.
- v4 chưa được kiểm tra kỹ trong kịch bản `kalloc()` thứ hai thất bại (OOM). Cần viết test case để đảm bảo không memory leak.
- Chưa đo throughput với `CPUS=2` để thấy lợi ích tách cache line của v4 trong môi trường multi-core.

(b) List of tasks to carry out in the next period:

1. Triển khai **v6 – Priority Boost**: thêm trường `ipc_boost` vào `struct proc`, scheduler ưu tiên tiến trình vừa wakeup từ pipe sleep. *Liên hệ lý thuyết: multi-level feedback queue, IO-bound boosting (Linux CFS wakeup preemption, Solaris priority).*
2. Triển khai **v7 – Cache-line Alignment**: thêm padding 64 byte giữa `nwrite` và `nread` trong `struct pipe`, đặt hai biến lên hai cache line khác nhau, loại bỏ false sharing. *Liên hệ lý thuyết: MESI protocol, `__cacheline_aligned` trong Linux kernel.*
3. Benchmark v4 với `CPUS=2` để đánh giá lợi ích tách cache line trong multi-core.
4. Chạy benchmark đầy đủ tất cả 7 phiên bản (+ ablation), vẽ biểu đồ so sánh tổng thể.

5. Phân tích nguyên nhân tăng/giảm hiệu năng giữa các phiên bản, gắn với mô hình Amdahl.

(c) Milestone:

- Hoàn thành v6 và v7 với đầy đủ lý thuyết, sơ đồ, và kết quả benchmark.
- Có biểu đồ so sánh đầy đủ 7 phiên bản (v1–v7).